山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202200130077	姓名：薛润泽	班级：人工智能
实验题目：实验 9 二叉树操作		
实验学时：2	实验日期： 2023.11.15	

实验目的：

练习实现二叉树，并实现递归与非递归的前序、中序、后序、层次遍历操作，实现计算二叉树高度和结点树。实现通过已知前序遍历结果和中序遍历结果来得到树，并输出后序遍历结果。

软件开发工具：

DEV C++

1. 实验内容

A:二叉树基础

创建二叉树类。二叉树的存储结构使用链表。提供操作：前序遍历、中序遍历、后序遍历、层次遍历、计算二叉树结点数目、计算二叉树高度，其中前序遍历要求以递归方式实现，中序遍历、后序遍历要求以非递归方式实现。

B:二叉树遍历

接收二叉树前序序列和中序序列(各元素各不相同)，输出该二叉树的后序序列。

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

A：二叉树基础

数据结构与算法：二叉树，栈，队列，数组

整体思路描述：

（1）建树过程：使用一个 Node*类型的数组来辅助进行链表建树，将该数组看做一个字典，每次遍历到相应序号的时候，都可以立刻映射出相应序号的节点指针。

（2）前序遍历、高度、节点数目计算：使用递归进行前序遍历，同时在前序遍历的时候同时完成对该节点高度、作为根时的树的总节点数目的计算。高度 = 左右子树最高的高度+1，总节点数目 = 左右子树数目之和 + 1。

（2）中序遍历与后序遍历：使用数据结构栈来辅助进行遍历。例如中序遍历，定义一个栈，首先将根节点压入栈，当栈不为空时持续进行操作。当该元素的左右孩子指针都为空的时候输出该元素值，并跳至下一次循环。如果该元素右孩子指针不为空，则将右孩子压入栈。建造一个新节点，将该节点的值赋给新节点，新节点的左右孩子指针都设置为空，将该节点压入栈。如果该节点的左孩子指针不为空，则将左孩子压入栈。按照先入后出原则，则对于一个节点和它的左右孩子，最后会以左孩子、该节点、右孩子的顺序输出。

（3）层次遍历：使用数据结构队列来辅助遍历，当队列不为空的时候持续操作。首先将根节点压入队列中，如果队首元素的元素左孩子不空，将左孩子入队。右孩子不空，将右孩子入队，最后输出该元素的值。

B：二叉树遍历

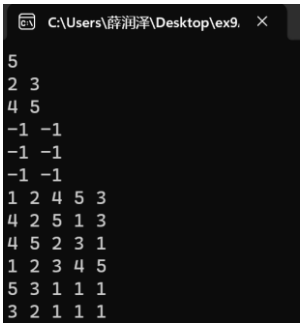
数据结构与算法：二叉树，前序、中序、后序遍历

整体思路描述：根据二叉树的前序、中序遍历的结果来将相应二叉树唯一确定出来。我选择在建树时每次插入树的节点是按照前序遍历顺序出现的节点。通过观察前序、中序遍历

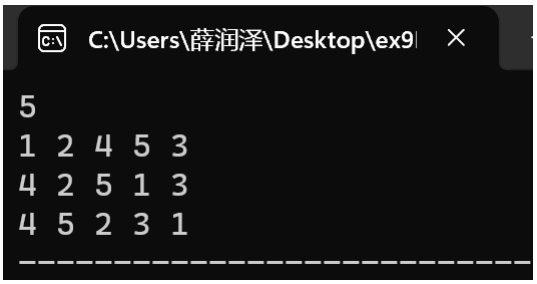
可以发现，建立二叉树可以由两个结论来进行。第一个结论是上一次操作的节点与本次操作的节点在中序遍历数组中的相对位置决定本次节点插入为左孩子还是右孩子，如果本次节点的相对位置在上一次操作的节点的左方，则作为左孩子插入到目的根中，如果在上一次操作的节点的右方，则作为右孩子插入到目的根中。第二个结论是当确认出插入为左孩子或右孩子后，插入的目的根是该节点右方或左方第一个遇到的已在树中且对应孩子指针为空的节点。比如确认了本次操作节点是以左孩子的形式插入到目的根中，我们从本次操作节点的右侧寻找目的根，要求该根是离本次操作节点最近的已经出现在了树中且该根的左孩子指针为空的节点。

3. 测试结果（测试输入，测试输出）

A：二叉树基础



B:二叉树遍历



4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

最终结果与预期相同

在解决 A：二叉树基础的过程中，最开始出现了 TLE 100ms 的情况，执行的时间超出了限定时间的 10 倍，最终找错误发现是类的析构函数出现了逻辑错误，导致相应实例没有正常析构，导致了 TLE。通过改正逻辑错误，最终解决了 TLE 的问题。

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

A：二叉树基础

```
#include<iostream>
struct Node{//树上节点
    Node(){
    leftptr = nullptr;
    rightptr = nullptr;
    return;
```

```
    };
    Node(int v):value(v){
    leftptr = nullptr;
    rightptr = nullptr;
    height = 1;
    sumNode = 1;
    return ;
    };
    int value;//记录节点 value
    int height;//记录该节点高度
    int sumNode;//记录以该节点为根的树的节点个数
    Node* leftptr = nullptr;
    Node* rightptr = nullptr;
};


template<typename T>
class Stack{//栈类
public:
    Stack(){
        element = new T[100000];
        listSize = 0 ;
        head = 0;
        tail = 0;
    }
    ~Stack(){delete []element;}
    bool empty()const{return listSize == 0;}
    void push(T& a){
        tail++;
        element[tail] = a;
        listSize++;
    }
    void pop(){
        if(!empty()){
            tail--;
            listSize--;
        }
    }
    T& top(){
        return element[tail];
    }
    void clear(){//清空栈
        if(!this->empty()){
            this->pop();
        }
    }
```

```
    void OutStack(){
        while(!this->empty()){
            std::cout<<this->top();
            this->pop();
        }
    }
private:
    T *element = nullptr;
    int listSize = 0;
    int head = 0;//头是栈的第一个元素索引，一般不进行操作
    int tail = 0;//尾是数组尾,随着元素压入、弹出栈而不断改变
};
template<typename T>
class Queue{//循环队列，实现 push，pop，top 等功能，先入先出
public:
    Queue(int n){//构造函数
        qBack = 0;
        qFront = 0;
        arrayLength = n + 1;
        element = new T[n + 1];//申请 n+1 个空间，因为循环队列数组中要空一个
    }
    ~Queue(){
        delete []element;
    }
    int Size(){//判断队列中元素数量
        return  (arrayLength + qBack - qFront)%arrayLength;//模 arrayLength 保证 qBack 在 qFront 右方时计算结果不超出 arrayLength，
    }
    bool empty(){//判断队列是否为空
        return qBack == qFront;//当 qBack = qFront 时为空，(qBack + 1)%arrayLength = qFront 时为满
    }
    void push(T a){
        if((qBack + 1)%arrayLength == qFront){//判断队列是否满
            std::cout<<"FULL QUEUE"<<'\n';
            return;
        }
        qBack = (qBack + 1)%arrayLength;
        element[qBack] = a;
    }
    T top(){
        if(this->empty()) std::cout<<"empty"<<'\n';
        return element[(qFront + 1)%arrayLength];
    }
    void pop(){
        qFront = (qFront + 1)%arrayLength;//弹出操作，即将 qFront 向前移动 1，模 arrayLength
```

```
确保 qFront 不超过
    }
private:
    T *element = nullptr;//元素数组指针
    int qBack;//记录队尾,随着元素入队而移动
    int qFront;//记录队首，随着元素出队而移动
    int arrayLength;//记录数组大小
};


class binaryTree{//二叉树类
public:
    binaryTree(){
        int n = 0; std::cin>>n;
        Nodenum = n;
        root = new Node(1);
        h = new Node*[n + 1];//h 是一个数组，实现 O(1)复杂度寻找相应序号的节点值
        h[1] = root;//将根节点赋给 h[1]
        int count = 1;//用于记录序号
        int a, b;
        while(count <= n){//当没有输入完 n 组数据时持续循环
            std::cin>>a>>b;//输入第 n 号节点的左右子节点值
            Node* temp = h[count];//将相应第 count 号的节点从 h 数组中取出
            if(a != -1){//如果插入的左节点不为空
                Node* cntNode = new Node(a);
                temp->leftptr = cntNode;//插入左节点
                h[a] = cntNode;//将新节点放入到 h 数组中相应位置
            }
            if(b != -1){//如果插入的右结点不为空
                Node* cntNode = new Node(b);
                temp->rightptr = cntNode;//插入右结点
                h[b] = cntNode;//将新节点放入到 h 数组相应位置
            }
            count++;
        }
        return;
    }
    void deleteOrder(Node* cnt){//用于析构函数删除 new 出的节点
        if(cnt->leftptr == nullptr && cnt->rightptr == nullptr) delete cnt;
        if(cnt->leftptr != nullptr) deleteOrder(cnt->leftptr);
        if(cnt->rightptr != nullptr) deleteOrder(cnt->rightptr);
        delete cnt;
    }
    ~binaryTree(){
        for(int i = 1; i <= Nodenum; i++){
            delete h[i];
```

```
        }
        delete h;
    }
    void PreOrder(Node* cntptr){//前序遍历,顺便完成每个节点高度和作为根的子树节点总数的计算
        if(cntptr == nullptr) return;
        std::cout<<cntptr->value<<' ';//输出该节点
        PreOrder(cntptr->leftptr);//遍历左子树
        PreOrder(cntptr->rightptr);//遍历右子树
        int maxheight = 0;//对高度进行计算 ,高度等于该节点左右子树中最高的那一棵 + 1
        if(cntptr->leftptr != nullptr) maxheight = (cntptr->leftptr)->height >maxheight?(cntptr->leftptr)->height : maxheight;
        if(cntptr->rightptr != nullptr) maxheight = (cntptr->rightptr)->height >maxheight?(cntptr->rightptr)->height : maxheight;
        cntptr->height = maxheight + 1;
        int maxNode = 1;//对总节点数进行计算，总节点数等于该节点左右子树中节点数之和 + 1
        if(cntptr->leftptr != nullptr) maxNode += (cntptr->leftptr)->sumNode;
        if(cntptr->rightptr != nullptr) maxNode +=(cntptr->rightptr)->sumNode;
        cntptr->sumNode = maxNode;
        return;
    }
    void preOrder(){
        PreOrder(root);
    }
    void inOrder(){//中序遍历
        Stack<Node> s;//使用一个栈
        s.push(*root);//将根节点入栈
        while(!s.empty()){//当栈不为空时持续循环操作
            Node temp = s.top();
            s.pop();
            if(temp.leftptr == nullptr && temp.rightptr == nullptr){//如果弹出元素的左孩子为空并且右孩子为空
                std::cout<<temp.value<<' ';//输出该元素值
                continue;
            }
            if(temp.rightptr != nullptr) s.push(*temp.rightptr);//如果该元素右孩子不为空,将右孩子压入栈
            Node outtemp(temp.value);//outtemp 用来输出，主动设置一个左右子树都为空，value 值为当前节点的 Node，并将其压入栈中
            s.push(outtemp);//新结点入栈
            if(temp.leftptr != nullptr) s.push(*temp.leftptr);//如果该元素左孩子不为空，将左孩子压入栈
        }
    }
    void postOrder(){//后序遍历,与后序遍历栈操作的顺序区别在于，先将新构造的左右孩
```

```
子都为空的节点压入栈,再将右孩子压入栈，最后将左孩子压入栈
        Stack<Node> s;
        s.push(*root);
        while(!s.empty()){
            Node temp = s.top();
            s.pop();
            if(temp.leftptr == nullptr && temp.rightptr == nullptr){
                std::cout<<temp.value<<' ';
                continue;
            }
            Node outtemp(temp.value);//outtemp 用来输出，左右子树都为空
            s.push(outtemp);
            if(temp.rightptr != nullptr) s.push(*temp.rightptr);
            if(temp.leftptr != nullptr) s.push(*temp.leftptr);
        }
    }
    void levelOrder(){//层次遍历
        Queue<Node*> q(100000);
        q.push(root);
        while(!q.empty()){//队列不为空时持续操作
            Node* temp = q.top();//取出队首元素
            q.pop();
            if(temp->leftptr != nullptr) q.push(temp->leftptr);//如果该元素左孩子不为空，将左
孩子入队
            if(temp->rightptr != nullptr) q.push(temp->rightptr);//如果该元素右孩子不为空，
将右孩子入队
            std::cout<<temp->value<<' ';//输出该元素值
        }
    }
    void Outheight(){//输出高度
        for(int i = 1; i <= Nodenum; i++){
            Node* temp = h[i];
            std::cout<<temp->height<<' ';
        }
    }
    void OutsumNode(){//输出节点总数
        for(int i = 1; i <= Nodenum; i++){
            Node* temp = h[i];
            std::cout<<temp->sumNode<<' ';
        }
    }
private:
    Node* root = nullptr;//根节点
    Node** h;//一个数组，直接映射树中元素
    int Nodenum = 0;//树的总节点个数
};
```

```
int main(){
    binaryTree tree;
    tree.preOrder(); std::cout<<'\n';
    tree.inOrder(); std::cout<<'\n';
    tree.postOrder(); std::cout<<'\n';
    tree.levelOrder(); std::cout<<'\n';
    tree.OutsumNode();std::cout<<'\n';
    tree.Outheight();std::cout<<'\n';
    return 0;
}
```

B：二叉树遍历

```
#include<iostream>

struct Node{
    Node(){};
    Node(int v){
        value = v;
        leftptr = nullptr;
        rightptr = nullptr;
    }
    int value;
    Node* leftptr = nullptr;
    Node* rightptr = nullptr;
};
class binaryTree{
public:
    //总结两句结论 1  上一次与本次的相对位置决定是左右节点中的哪个
    // 2  跟随方向，加入到最近的空点中
    binaryTree(){
        int n = 0;
        std::cin>>n;
        num = n;
        preOrder = new int[n + 1];
        inOrder = new Node*[n + 1];
        sym = new bool[n + 1];
        Orderin = new int[n + 1];
        for(int i = 1; i <= n; i++){
            std::cin>>preOrder[i];
        }
        for(int i = 1; i <= n; i++){
            int value ; std::cin>>value;
            inOrder[i] = new Node(value);
```

```
            Orderin[value] = i;//该数组索引为值，值为 inOrder 索引
            sym[i] = 0;//代表未入树

        }
        root = inOrder[Orderin[1]];
    }
    ~binaryTree(){
         delete []preOrder;
        delete []sym;
        delete Orderin;//记录相应中序遍历元素的相应位置
        for(int i = 1; i <= num; i++){
            delete inOrder[i];
        }
        delete []inOrder;
    }
    void buildTree(){
        //Orderin 记录相应的值在 inOrder 中的索引，意义为索引
        //inOrder 为一个 Node* 矩阵，保存着所有的节点
        //sym 判断对应索引位置是否入树
        //preOrder 是 int 类型的前向遍历的结果矩阵
        sym[Orderin[1]] = 1;
        int last;//记录上一次操作的节点在 inOrder 中的索引
        for(int i = 2; i <= num; i++){
            int last = Orderin[preOrder[i - 1]];//last 是上一次遍历元素的位置
            int temp = Orderin[preOrder[i]];//temp 是现在遍历到的元素在 inOrder 中的位置
            if(temp < last){//插入左节点
                sym[temp] = 1;
                int move = temp + 1;
                while(sym[move] == 0 || (sym[move] == 1 && inOrder[move]->leftptr!=
nullptr)) move++;//如果遍历到的点没有出现在树中，或者该点左孩子指针不为空则移动，直到找到第一个出现在树中且左孩子指针为空的点
                //当结束遍历时，说明找到了相应点
                inOrder[move]->leftptr = inOrder[temp];//连接
            }
            else if(temp > last){//插入右节点
                sym[temp] = 1;
                int move = temp - 1;
                while(sym[move] == 0 || (sym[move] == 1 && inOrder[move]->rightptr!=
nullptr)) move--;//如果遍历到的点没有出现在树中，或者该点右孩子指针不为空则移动，直到找到第一个出现在树中且右孩子指针为空的点
                inOrder[move]->rightptr = inOrder[temp];//连接
            }
        }
    }
    void PostOrder(){
        this->postOrder(root);
```

```
    }
    void postOrder(Node* cnt){//递归实现后序遍历
        if(cnt == nullptr) return;
        if(cnt->leftptr != nullptr) {
        postOrder(cnt->leftptr);}
        if(cnt->rightptr != nullptr) {
        postOrder(cnt->rightptr);}
        std::cout<<cnt->value<<' ';
    }

private:
     int *preOrder;
     Node **inOrder;
     bool * sym;//用于判断相应位置元素是否在树中,0 为空，1 为存在
     int *Orderin;//记录相应中序遍历元素的相应位置
     Node* root;
     int num = 0;
};
int main(){
    binaryTree tree;
    tree.buildTree();
    tree.PostOrder();
    return 0 ;
}
```